

Informe tecnico de despliegue

Pipeline, aceleracion y despliegue del Sistema Centinela en el borde

Redes Neuronales — Deep Learning . Maestria en Ciencia de Datos, Universidad Santo Tomas
Grupo: Juan Manuel Gomez . Camilo Andres Martinez . Sergio Beltran

En una frase. La Fase 3 industrializa el sistema de las fases anteriores: un pipeline de datos comparado en ambos frameworks, entrenamiento en GPU (RTX 4050) con precision mixta y checkpointing con reanudacion, un modelo comprimido a INT8 (35 KB) verificado en tres formatos, y una ruta de despliegue concreta — la rama visual offline en la tablet del inspector y la rama temporal por API REST en la nube.

1. Comparativa de pipelines (tf.data vs DataLoader)

El mismo trabajo — leer 1.890 huellas desde carpetas *train/val/test*, normalizar, barajar y hacer lotes de 64 — se implemento en ambos frameworks midiendo el tiempo por epoca. El objetivo de todo pipeline es evitar la **GPU starvation** —que la GPU quede ociosa esperando datos— solapando la preparacion del lote siguiente con el computo del actual; por eso prefetch va siempre al final del orden canonico (cargar, map, cache, shuffle, batch, prefetch). En *tf.data*, **cache + prefetch(AUTOTUNE)** paga el llenado del cache en la epoca 1 (1,05 s) y desde la epoca 2 lee de memoria: **0,03–0,05 s/epoca, una mejora de ~4×** frente al pipeline sin optimizar (0,16–0,35 s). En PyTorch, medido en la GPU (RTX 4050): el DataLoader sin workers rinde ~0,40 s/epoca, mientras que con workers optimos la primera epoca paga el arranque de los procesos (~12 s) y luego se estabiliza en **0,23 s/epoca** — un hallazgo medido que matiza la receta estandar: los workers pagan a partir de la segunda epoca, cuando el arranque en frio ya se amortizo. Los dos runtimes se aislaron en procesos separados, siguiendo la advertencia de la ficha sobre su co-carga.

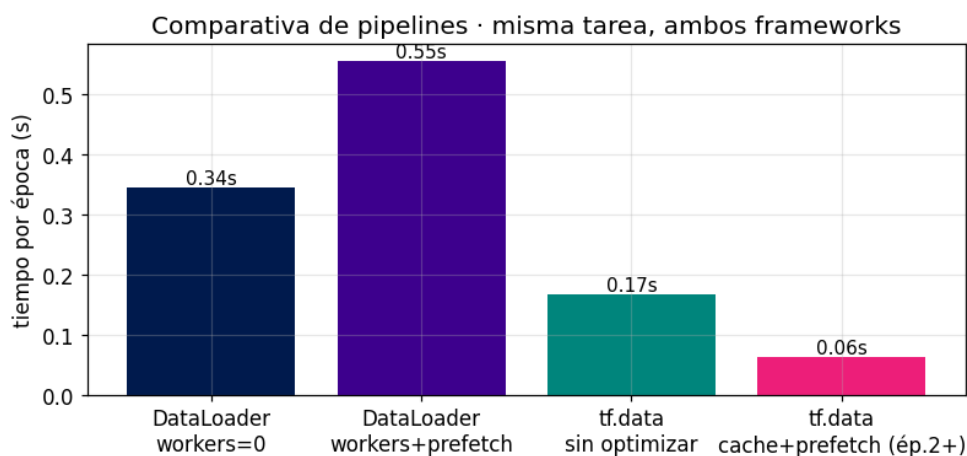


Figura 1. Tiempo por epoca de la misma tarea en ambos frameworks, con y sin optimizaciones.

2. Data Augmentation con semantica y rendimiento del entrenamiento

La huella es una **reticula calendario** (filas=anos, columnas=meses), lo que invierte el catalogo clasico: se **rechazaron** HorizontalFlip (espejar las columnas invierte el tiempo y cambia la etiqueta real) y Rotation/ResizedCrop (destruyen la posicion ano×mes, que es la semantica). Se aplicaron 4 aumentos que la respetan — jitter suave de brillo/contraste, desenfoque leve, desfase horizontal $\leq \frac{1}{2}$ mes y **borrado aleatorio** (el analogo exacto de las lecturas faltantes de la base) — mas **Mixup** como tecnica avanzada. Resultado medido: con-aumento y sin-aumento empatan en el test limpio (0,891 vs 0,901, diferencia ~1 pt); para produccion se eligio el modelo **con aumento**, porque el borrado aleatorio lo entreno a decidir con lecturas faltantes — una robustez de campo que el test limpio no mide.

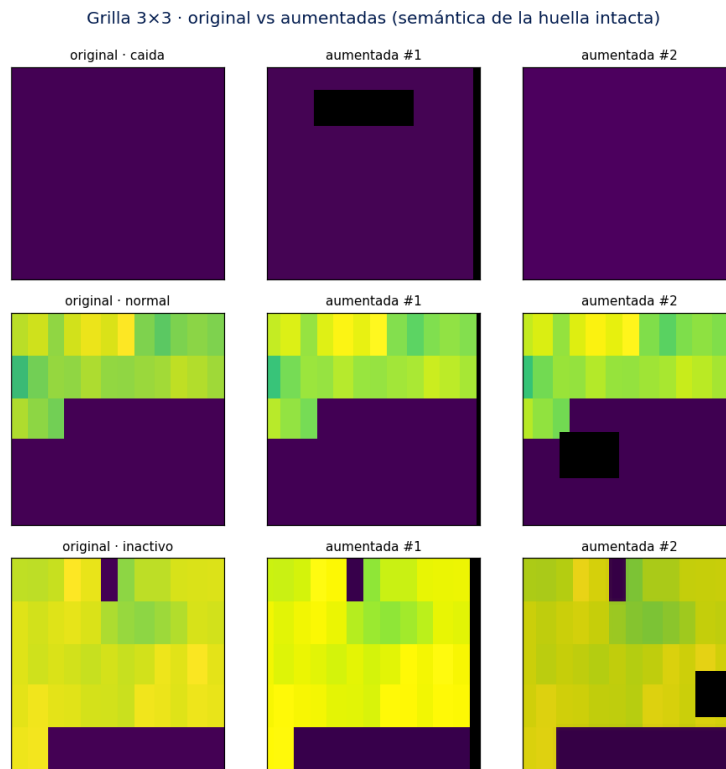


Figura 2. Grilla original vs aumentadas: la estructura calendario permanece intacta.

Entrenamiento acelerado. El modelo del borde es **CentinelaEdgeNet**, una CNN compacta de **27.939 parámetros** (81× menos que el backbone MobileNetV2 de la Fase 2, que tiene 2,26 M —ambas cifras calculadas en el cuaderno, no citadas de memoria) entrenada desde cero: costo de diseño de −4,0 pt de exactitud (0,891 vs 0,931) a cambio de un modelo distribuido offline. El bucle usa **precision mixta device-aware** (FP16 + GradScaler en GPU) y **checkpointing cada 5 épocas con reanudación**, verificada en la ejecución. **Medición en GPU real (NVIDIA RTX 4050 Laptop, 6 GB):** con FP16 la **VRAM pico baja de 75 a 55 MB (−26 %)**, exactamente el ahorro que la teoría predice en las activaciones. El **tiempo por época, en cambio, no mejora** (2,86 s en FP32 vs 3,41 s con FP16): es un resultado medido, no asumido, y tiene explicación — con un modelo de solo 27.939 parámetros e imágenes de 64 px, el trabajo por lote es tan liviano que el *overhead* de convertir entre FP16 y FP32 pesa más que lo que ahorran los Tensor Cores. La aceleración temporal de FP16 se materializa en modelos grandes (ResNet, Transformers), no en una red *edge* diminuta; el beneficio que sí aplica aquí —y siempre— es el de memoria. Lo reportamos así, con escepticismo hacia lo que "debería" acelerar, en vez de afirmar una mejora que no ocurrió (NVIDIA, 2023).

3. Optimización Edge: tabla antes/después

Compresión elegida: **cuantización post-entrenamiento a INT8** del grafo ONNX completo (Jacob et al., 2018) — la vía directa hacia runtimes de borde (ONNX Runtime Mobile / LiteRT).

Métrica	Antes (FP32)	Después (INT8)	Efecto
Tamaño del modelo	111 KB	35 KB	−68 %
Latencia (ms/imagen, x86 de pruebas)	0,25	2,67	ver trade-off
Exactitud (test, 3 clases)	0,893	0,891	−0,2 pt

Trade-off. El tamaño cae 68 % perdiendo 0,2 pt de exactitud — ideal para distribuir actualizaciones offline a decenas de tablets. La latencia INT8 *empeora en el x86 de pruebas* (ORT usa un kernel genérico para convoluciones cuantizadas en esta plataforma); en el hardware objetivo ARM (XNNPACK/NNAPI) el INT8 típicamente acelera. Ambas latencias están >30× por debajo del presupuesto de campo (100 ms/huella), así que el criterio dominante es el tamaño: **aceptable para el uso offline**.

Exportacion verificada en tres formatos. .pth (PyTorch), .onnx (puente portable, por la ruta estable `dynamo=False`) y .keras (implementacion equivalente en Keras con los pesos transferidos — como subraya la ficha, dos implementaciones, no el mismo objeto). Paridad verificada con `np.allclose(atol=1e-4)`: `torch↔onnx` dif. max. $2,2e-06$; `torch↔keras` $9,5e-07$. Un modelo que predice distinto tras exportarse no es desplegable; esta verificacion cierra el ciclo.

4. Ruta de despliegue y justificacion de ingenieria

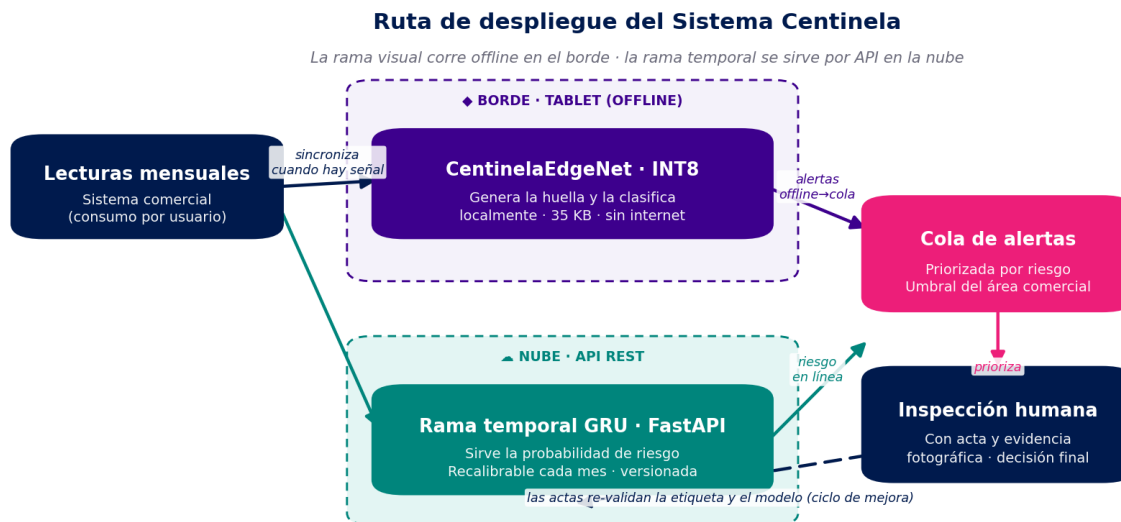


Figura 3. Ruta de despliegue: rama visual offline en el borde; rama temporal por API REST en la nube; las actas re-validan el ciclo.

Tablet (borde, offline). La app genera la huella localmente desde las lecturas sincronizadas y ejecuta `centinela_edgenet_int8.onnx` (35 KB) con ONNX Runtime Mobile, o su conversion a LiteRT via el modelo Keras en Android. Sin conectividad, la inferencia es local y las alertas se encolan hasta recuperar señal — la restriccion de campo que guio toda la fase. **Nube (API REST).** La GRU temporal de la Fase 2 se sirve con FastAPI (demo ejecutada en el cuaderno: caida sostenida → prob. 0,99, alerta; consumo estable → 0,01). Dos razones de ingenieria la mantienen en la nube: la exportacion de capas recurrentes a runtimes de borde es poco fiable (operadores GRU/LSTM con soporte variable), y el modelo temporal se recalibra con cada mes de lecturas — versionarlo en un punto evita reflashear tablets.

Validez de la etiqueta (nota de integridad). La etiqueta actual se deriva de una regla determinista sobre el mismo consumo que ven las redes — una **circularidad** que declaramos abiertamente: por eso un ROC-AUC alto recupera en parte una regla conocida, y no basta como prueba de deteccion. El aporte legitimo del modelo esta en la **zona gris** (~1.860 usuarios donde la regla es fragil: con $\pm 10\%$ de ruido ~184 etiquetas cambian), donde aprende una frontera suave. La validacion definitiva son las **actas de inspeccion reales**, independientes del consumo; hasta entonces se despliega la capacidad tecnica, no una recomendacion de campo sin verificar.

Despliegue responsable (Transparencia y Beneficio Social). El sistema es un *priorizador*, no un juez. Mitigacion de falsos positivos: ninguna alerta corta el servicio ni sanciona automaticamente — toda alerta pasa por inspeccion humana con acta y evidencia fotografica, y el umbral lo fija el area comercial segun su capacidad. Mitigacion de falsos negativos: se conserva el muestreo aleatorio de inspecciones (el modelo prioriza, no reemplaza el control). Transparencia: cada alerta registra la version del modelo y las senales que la dispararon, y el desempeno se re-valida contra las actas de campo en cada ciclo. Equidad: se monitorea que las alertas no se concentren injustificadamente por zona o estrato.

Trazabilidad de cifras: todos los conteos de parametros (EdgeNet 27,939; MobileNetV2 2,26 M; ResNet50 23,6 M), tiempos, tamanos y metricas de este informe se **calculan e imprimen** en el cuaderno adjunto — no hay valores escritos a mano, de modo que texto y ejecucion siempre coinciden.

Referencias (APA 7)

- Howard, A., Sandler, M., Chu, G., et al. (2018). *MobileNetV2: Inverted residuals and linear bottlenecks*. Proceedings of the IEEE CVPR, 4510–4520.
- Jacob, B., Kligys, S., Chen, B., et al. (2018). *Quantization and training of neural networks for efficient integer-arithmetic-only inference*. Proceedings of the IEEE CVPR, 2704–2713.
- NVIDIA. (2023). *Train with mixed precision: User guide*. NVIDIA Developer Documentation.
- Paszke, A., Gross, S., Massa, F., et al. (2019). *PyTorch: An imperative style, high-performance deep learning library*. Advances in NeurIPS, 32.
- Zheng, Z., Yang, Y., Niu, X., Dai, H., & Zhou, Y. (2018). *Wide and deep convolutional neural networks for electricity-theft detection to secure smart grids*. IEEE Transactions on Industrial Informatics, 14(4), 1606–1615.

Documento academico. Datos anonimizados (Ley 1581 de 2012). Elaborado con apoyo de IA generativa, documentado en la Bitacora de IA adjunta. Notebook ejecutado, modelos exportados (.pth/.keras/.onnx fp32+int8) y demo de API acompanan la entrega.